

The TSP asks us to determine the order of travel between cities that will minimize travel costs. Like all the best problems in science, it's easy to state and extremely difficult to solve.

Setting Up the Problem

Let's fire up Python and start exploring. First, we'll randomly generate a map for our salesman to traverse. We start by selecting some number N that will represent the number of cities we want on the map. Let's say $N = 40$. Then we'll select 40 sets of coordinates: one x value and one y value for each city. We'll use the `numpy` module to do the random selection:

```
import numpy as np
random_seed = 1729
np.random.seed(random_seed)
N = 40
x = np.random.rand(N)
y = np.random.rand(N)
```

In this snippet, we used the `numpy` module's `random.seed()` method. This method takes any number you pass to it and uses that number as a “seed” for its pseudorandom number generation algorithm (see Chapter 5 for more about pseudorandom number generation). This means that if you use the same seed we used in the preceding snippet, you'll generate the same random numbers we generate here, so it will be easier to follow the code and you'll get plots and results that are identical to these.

Next, we'll zip the x values and y values together to create `cities`, a list containing the coordinate pair for each of our 40 randomly generated city locations.

```
points = zip(x,y)
cities = list(points)
```

If you run `print(cities)` in the Python console, you can see a list containing the randomly generated points. Each of these points represents a city. We won't bother to give any city a name. Instead, we can refer to the first city as `cities[0]`, the second as `cities[1]`, and so on.

We already have everything we need to propose a solution to the TSP. Our first proposed solution will be to simply visit all the cities in the order